Cascade Lift Driver-Control Plan

Team 936C ▪ VEX V5RC *Override* 2026–27

Reference plan — not the implementation plan.

At a glance

- The lift runs on **count-driven presets**. The **rung** (which object you place next) auto-advances *only* after a confirmed placement.
- The driver selects the **goal class**; that sets the base height. The rung sets the step up the stack.
- Manual controls stay for two distinct jobs: **fine jog** when a preset is off, and **rung sync** when adding to a stack you did not build.
- **No distance sensor, no vision.** Software tracking plus manual sync only.

1 Purpose and scope

This is the agreed design for how the driver commands the cascade lift during the Driver Controlled Period. It is a reference to return to – not a build specification. An implementation plan comes later.

In scope	Driver control of the cascade lift: presets, button map, auto-advance, sync, recovery.
Out of scope	Autonomous routines, and any sensing of the stack (distance, vision, pose).
Constraint	No new hardware beyond the existing lift encoders and the homing switch.

2 What changes with a driver-control focus

Autonomous is deterministic: it knows exactly what it places, in what order, on which goal. Driver control does not. Every piece of complexity below exists only to absorb what the driver *cannot* know – pre-existing stacks, missed placements, defense, and human error.

Layer	Used by	Responsibility
Stacker (subsystem)	auton & driver	Homing, linear degree-to-inch conversion, PIDF hold, preset table, `go_to_stack(class, rung)`, `go_to_travel()`
LiftControl (control/)	**driver only**	Active goal class, rung tracking, auto-advance, sync, button map, readout, endgame lockout
ScoreCommand	**driver only**	One-button raise, deposit, retract, lower macro

Single source of truth

Both paths share the same preset table. Calibrating a height fixes the driver and autonomous behaviour together – there is never a second copy of "what height is rung 3 on a neutral goal."

3 Driver control — the loop

The driver's whole job at a goal is: **align, press R1, repeat.** Everything else is state that gets set while stationary.

> **Ergonomics rule**
> The driver's thumbs live on the sticks. Scoring is inherently a stationary action, so face buttons and the D-pad are fine – but the one button pressed *in motion* has to be a trigger.

Button	Action	When
R1	**Score.** Raise to rung → dwell → deposit → retract → return to travel. Advances the rung on a confirmed place.	Aligned at goal (trigger)
R2	**Retry.** Same rung, no advance.	After a miss
X	**New stack.** Sets `rung = 0`: "this goal is empty."	On arrival at a fresh goal
Y	**Cycle goal class.** Alliance → Neutral short → Neutral tall. Does *not* reset the rung.	On arrival
Up / Down	**Rung sync** ± 1, to match a stack you did not build.	Stationary
L1 / L2	**Fine jog** up / down (hold). Captures the position as its hold target on release.	Recovery only
A	**Home / re-zero** the lift.	Recovery only
B	Cancel macros (unchanged).	Any time

Two behaviors worth building deliberately:

- **R1 is fire-and-forget.** The sequence runs in the background via `CommandController::poll()`, so the driver can start driving the moment the object is deposited – they do not wait for the lift to finish lowering.
- **Auto-lower immediately after deposit.** Return to travel height right away. A raised cascade is a sail in defense and raises the center of gravity.

4 The rung model

Keep the rung **per goal class with an explicit reset** – not per-goal memory.

- There are seven legal scoring targets: two alliance goals, four neutral short goals, and one center goal.
- Selecting one of seven with buttons is worse than pressing **X** once on arrival. X is a single, teachable habit that is *always* correct.
- Per-goal memory only pays off if the driver constantly returns to partially built goals. If that turns out to be true, the fix is pose-based goal identification – which also makes X disappear.

Object parity is fixed by <SC2> and the software always knows it: **odd rung = Pin, even rung = Cup.** The readout therefore always says what to load. Combined with <SG6> (one Pin *and* one Cup), the driver can place both in a single stop: R1 (pin), the rung auto-advances to the cup, R1 again.

5 Height model

Objects nest, so each added object raises the stack by **at most one half (3.25 in)** – never the full 6.5 in – and the goal rim shifts the whole stack one-to-one.

```
release_height(class, rung) = rim[class] + (rung − 1) ∗ STEP + CLEARANCE
```

`STEP = 3.25` to start. **This is a seed, not truth.** Pins and cups may seat differently, so store a calibrated 3×6 array initialized from this formula and overwrite the entries you measure. Cap the maximum rung so the lift always stays under the 50 in height limit (`<SG3>`).

6 Auto-advance trigger

Advance **only on a confirmed placement**, at the moment of deposit.

- **Pins:** the intake's `has_pin()` goes true → false. That is a sensor you already have.
- **Cups:** no sensor yet, so gate on the macro completing without a timeout. This is weaker; accept it and rely on R2 for misses.

If `has_pin()` never clears, no advance happens. That is the correct failure mode: the driver presses R2 instead of silently building a stack that is not there.

7 Failure playbook

Situation	What the driver sees	Action
Pin did not seat / fell	deposit timed out, no advance	**R2 retry**
Pin still in intake	`has_pin` stays true, no advance	**L2 jog down, eject, re-approach, R2**
Rung out of sync (stack toppled, opponent added)	lift reaches too high or low	**Up / Down to match reality; X if actually empty**
Need a Cup but holding a Pin	readout says `load CUP`	**Use the cup already carried (`<SG6>`), else loader**
Wrong goal class selected	lift aims at the wrong height	**Y to cycle, then R1**
Lift stalls	no motion, motor stall	**A to home, then re-approach**
Defended mid-raise	–	**PIDF + brake holds position; drive out and retry**
Endgame, aiming at the center goal	error buzz, no movement	**Pick another class (`<SG12>`)**

The sync and reset buttons are **not** a fallback. On any neutral goal that a partner or opponent has touched, they are part of the normal workflow. That is the cost of running without sensors, and it is an acceptable trade for now.

8 Match-phase behavior

- **Start of `opcontrol()`:** reset to `rung = 0`, neutral-short class by default. Later, auton can seed the starting rung if the driver should continue the goal auton just used.
- **Endgame:** track elapsed time from the start of `opcontrol()` (PROS exposes no match clock) and refuse the center-goal class in the final 10 seconds.
- **Travel:** always return to travel height after a score, and home on A if anything looks wrong.

9 Autonomous side (deliberately minimal)

Autonomous calls `stacker().go_to_stack(class, rung)` with hardcoded values, or `go_to_height()` for non-scoring moves. It needs **no** `LiftControl`, no rung tracking, no sync, and no readout. It still homes first and still uses the same preset table. Seeding the starting rung at handoff is optional; a plain `rung = 0` start is fine.

10 Build order

1. **Homing** + linear degree-to-inch conversion (including the cascade stage multiplier) + wire `stacker().tick()` into `opcontrol()`. Nothing else works without this.
2. **Preset table** + `go_to_stack()` + `at_target()` — prove all fifteen cells land consistently.
3. **`LiftControl`** + button map + screen readout + rumble, driven by manual rung nudging only.
4. **`ScoreCommand`** with confirmed-place auto-advance, R2 retry, and auto-lower to travel.
5. **Endgame lockout** and match-time tracking.
6. **Practice-match calibration**, then a driver drill on the failure playbook.

11 Out of scope

Deliberately deferred

Distance-sensor stack-top detection · vision or AprilTag goal detection · pose-based goal selection · per-goal rung memory · automatic stack counting.

A Reference: goal geometry and rung heights

Pin and Cup are each 6.5 in tall, in two halves of 3.25 in. Rims: alliance 3.25 in, neutral short 5.80 in, center (tall) 8.70 in. Object parity is fixed: odd rung is a Pin, even rung is a Cup. Values below are `rim + (rung − 1)* 3.25`, that is, the release height above the floor.

Rung	Alliance	Neutral short	Center tall	Object
1	3.25 in	5.80 in	8.70 in	Pin
2	6.50 in	9.05 in	11.95 in	Cup
3	9.75 in	12.30 in	15.20 in	Pin
4	13.00 in	15.55 in	18.45 in	Cup
5	16.25 in	18.80 in	21.70 in	Pin

Rung 1 is only valid on an **empty** goal, because <SC2> allows a Goal to hold exactly one Pin half. Pressing X at a goal that already holds a pin commands a height where the opening is occupied, and the pin will not seat.

B Reference: scoring and key rules

- **Scoring:** autonomous bonus 12; each scored alliance-colored Pin half 5; each scored yellow Pin half 10; each robot in the Midfield 8.
- **Autonomous Win Point:** at least seven Pins scored for your alliance, and at least three goals each containing two Pins scored for your alliance.

Rule	What it means here
<SC2>	A Goal holds at most one Pin half; each half of a Cup holds at most one Pin half. This forces strict alternation: Goal → Pin → Cup → Pin → Cup . . .
<SC3>	Each *visible* half of a Placed Pin scores. A half hidden inside the opaque Cup half does not score.
<SG3>	No part of the robot may exceed 50 in height at any point – this caps the maximum rung.
<SG6>	Possession is limited to one Pin and one Cup, so carrying both is legal and avoids parity problems.
<SG9>	Opposing Alliance-colored Goals are protected. Valid targets are your own alliance goal and the neutral goals.
<SG10>	Placed objects cannot be removed from neutral Goals, but adding to them is allowed.
<SG12>	No placing Scoring Objects on the Midfield (center) Goal during the Endgame – the final 10 seconds.

The one honest limitation

Without sensors, the robot only knows what it built. Autonomous placements can be seeded exactly. A partner's placements and an opponent's additions cannot. The rung sync buttons are therefore a required part of the workflow on any neutral goal – not merely a fallback.